Anomaly Detection via the Isolation Forest Algorithm using Rare Bird Sighting Data

Dai Ngo Tan
Masters of Applied Data Science
University of Victoria
Victoria, BC, Canada
ntdai95@uvic.ca

Naomi Phan
Masters of Applied Data Science
University of Victoria
Victoria, BC, Canada
naomiphan@uvic.ca

Jeremy Perkins
Masters of Applied Data Science
University of Victoria
Victoria, BC, Canada
jperkins@uvic.ca

***Abstract*—The Isolation Forest algorithm provides a way to quickly find anomalous data points, bypassing the usual computationally expensive need to first process normal points. The algorithm is implemented in PySpark on a massive dataset, the eBird database, that comes with anomaly labels, providing a natural test of the algorithm on a massive dataset. Our experiment reached moderate accuracy, with AUC of 0.67. Upon closer inspection, the model only predicted anomalies in geographic locations where bird observations were less frequent. This highlighted the algorithm's sensitivity to skewed feature distributions (feature-specific swamping), where it ignores other relevant features, focusing only on the sparse range of that particular feature.**

Keywords—Isolation forest, massive dataset, eBird, PySpark, swamping, skewed feature distribution

I. Introduction

The isolation forest (iForest) algorithm's main goal is to detect anomalies directly, in contrast to previous approaches that define anomalies relative to normal points [1]. The premise of the algorithm is that tree structures will inherently tend to have anomalous data isolated at points higher in the tree, close to the root, whereas normal points will be isolated much lower in the tree, near the leaves. An ensemble of iTrees is built, with partitions randomly generated. Anomalies correspond to nodes with short average path length among the ensemble then.

The algorithm offers several advantages that make it very practical to use:

- Works well with high-dimensional data with many irrelevant attributes.
- The training data need not contain many anomalies.
- Convergence is quick, with a small number of trees.
- There is no need to completely construct all trees since anomalies are found higher in trees. Small sub-samplings will work well as a result.
- Avoids high computation cost of distance- or density-based methods.
- Linear time complexity and low memory requirement.

Standard methods usually profile what normal data points look like. Because of this, expensive calculations are needed to calculate distances between these normal points. In addition, because they are not optimized for detecting anomalies, they tend to have low accuracy in identifying those anomalies. On the other hand, iForest does not waste computational effort dealing with large amounts of normal data. It explicitly isolates the anomalies early on.

The algorithm was tested on a number of datasets that involved inherently numerical features. As a result, we chose a dataset that contains both numerical and categorical features: The eBird dataset. This dataset consists of more than two billion observations worldwide of a given species, at a specific location and time. While most datasets require some judgement to set a threshold on what points are anomalies, the eBird dataset comes with anomalies labeled. Observations that are unusual are auto-flagged to be reviewed by experts; the database contains an attribute to record these reviews, giving a natural ground-truth that doesn't require an arbitrary decision to be made.

The large size of the database makes it challenging to work with, requiring strategies to address processing speed. With the use of PySpark [2], we were able to manage processing of a dataset with about a million observations. Another strategy that is used to deal with large datasets is random sampling. While taking a random sample from a dataset can make processing times more feasible, care must be taken that the sample is done in a way that doesn't ruin the structure of the data. To address this issue, stratified sampling [3] allows a reduction of the dataset that respects the relative structure among relevant features. In this case, the species label is particularly relevant: When taking a subsample, since there are so many species in the dataset and we want to avoid changing species frequency (of

rare species, in particular), it is important to retain the same distribution of species.

This paper starts by giving a detailed summary of the Isolation Forest algorithm in Section II. The dataset is described in detail and our implementation of the algorithm is outlined in Section III. We mirrored the original paper in terms of measuring accuracy of the model and Section IV describes the experimental results. These results are discussed and interpreted in Section V, where we highlighted a particular shortcoming of the way the algorithm treats skewed feature distributions. Section VI is the conclusion.

II. Isolation Forest

Rather than constructing a profile of normal behavior and flagging deviations from it, iForest exploits two properties of anomalies: they are rare, and they are structurally different from most of the data points. Because of these two properties, anomalies are inherently easier to isolate. If we recursively partition a dataset using random splits, an anomaly will be separated from the rest of the data in very few steps, placing it close to the root of the tree. A normal data point is usually surrounded by a dense cluster, and so it will require significantly more partitioning steps before it is fully isolated.

By averaging the path lengths across a forest of randomly constructed trees, we obtain a stable estimate of isolation difficulty for each data point. Those with shorter average path lengths are flagged as potential anomalies. An anomaly score s is then computed for each data point, normalized between 0 and 1.

The algorithm has only two hyperparameters: the number of trees t and the sub-sampling size ψ. The original authors recommended a sub-sampling size of *256* and an ensemble of *100* trees for optimal performance, values validated through rigorous empirical testing.

The iForest algorithm consists of four steps. The first two constitute the training phase, in which a small random subset of the data is used to construct the forest. The last two form the evaluation phase, where the trained model is applied to all samples in the full dataset. Specifically: (1) a random subset is drawn based on the sub-sampling size, and a random tree is constructed through recursive partitioning; (2) this process is repeated t times to form the complete forest; (3) the average path length is computed for each data point across all t trees in the forest; and (4) all data samples, including those not seen during training, are passed through the forest, and an anomaly score is computed for each.

1) Random Data Partitioning and Tree Construction

In the training phase, we draw a small random subset of $\psi = 256$ samples from the full dataset. The decision to sub-sample rather than use the entire dataset is motivated by both efficiency and accuracy. From an efficiency standpoint, isolating 256 data points is orders of magnitude faster than operating on a dataset that may contain millions or billions of records. From an accuracy standpoint, random sub-sampling has been shown to improve outlier isolation. Because anomalies are rare by nature, a small random sample will typically contain only a handful of them, surrounded predominantly by normal instances. This disparity makes anomalies stand out even more, improving detection reliability.

Sub-sampling also directly addresses two well-known problems in anomaly detection: *swamping* and *masking*. Swamping occurs when an anomaly lies too close to a dense cluster of normal points, making it difficult to distinguish as unusual. Since sub-sampling may exclude some of those nearby normal points, the anomaly becomes easier to isolate. Masking, on the other hand, occurs when multiple anomalies cluster together, causing them to collectively resemble a group of normal instances. Sub-sampling draws only a small fraction of any such group, keeping the anomalies sparse and distinguishable from one another.

Once the subset is drawn, we recursively partition the data by randomly selecting an attribute and a split value drawn uniformly from the minimum-to-maximum range of that feature among the current node's data points. Since split values are drawn from continuous numerical ranges, iForest natively supports only numerical features. It is critical to emphasize that at every recursive step, a completely new feature and a completely new split value are selected independently – the algorithm has no memory of previous splits. Each node makes a fresh, independent, random decision.

The node's left child receives all data points with feature values below the split value, while the right child receives those above it, forming a Binary Search Tree structure. A tree built from a data set of size ψ has $2\psi - 1$ nodes, keeping the memory footprint low. This data partitioning recursively continues until one of three stopping conditions is met: (1) only a single data point remains in the node, meaning it has been perfectly isolated; (2) all remaining data points in a node have identical feature values and cannot be further partitioned; or (3) the tree reaches its maximum height, defined as:

$$\text{h_max} = [log_2(\psi)] \quad (1)$$

For the recommended $\psi = 256$, this gives a maximum depth of 8. This height limit is both practically and theoretically justified. Because anomalies are isolated near the root of the tree, the deeper layers contribute very little useful signal and serve only to increase computation. Once t trees have been constructed, the training phase is complete.

It is worth noting that the use of random partitioning is a deliberate design choice. Most traditional anomaly detection algorithms rely heavily on distance or density calculations between data points – computing how far each point sits from its neighbors or from some centroid of normal behavior. These calculations are inherently expensive, scaling poorly as the dataset grows. iForest sidesteps this entirely by using random partitioning instead. The result is an algorithm that is not only significantly faster, but in many cases more accurate as well, particularly in high-dimensional spaces where distance-based measures are known to become unreliable due to the curse of dimensionality.

2) Average Path Length

Since random partitioning introduces variance, a single tree would produce an unreliable and biased estimate of any data point's isolation depth. Building a forest of $t = 100$ trees and

averaging the results smooths out this variance and yields a stable, consistent estimate.

For a given data point x, the path length $h(x)$ in a single tree is the number of edges traversed from the root to the node where x terminates. However, because trees are stopped early at maximum depth, some terminal nodes may still contain multiple data points – meaning x was not fully isolated. Using the raw depth in such cases would underestimate the true isolation difficulty (in other words, the true path length), since additional splits would still have been needed. To correct this, an adjustment is applied based on the expected path length for a Binary Search Tree (BST) node containing n remaining points:

$$c(n) = 2H(n-1) - 2\frac{n-1}{n} \quad (2)$$

H(i) is the harmonic number, which can be estimated using Euler's constant:

$$H(i) \cong \ln(i) + 0.577 \quad (3)$$

The boundary conditions are $c(1) = 0$ (a single point is already perfectly isolated and needs no further splits) and $c(2) = 1$.

This formula is borrowed from the well-established result that the average number of comparisons in an unsuccessful BST search follows this harmonic relationship, which maps directly onto random recursive binary partitioning.

The adjusted path length for a point x that reaches depth d in a terminal node containing n remaining points is therefore:

$$h(x) = d + c(n) \quad (4)$$

The average path length across the full forest consisting of t trees is then computed as:

$$E[h(x)] = \left(\frac{1}{t}\right) * \sum_{i=1}^{t} h_i(x) \quad (5)$$

3) Anomaly Score Computation

In the evaluation phase, we pass the entire dataset through the forest that was constructed during training, including data points that were never seen during the sub-sampling process. For each data point, x, we compute its adjusted path length $h(x)$ in each of the t trees, then average these values to obtain $E[h(x)]$ as described above.

The anomaly score $s(x, n)$ is then computed as:

$$s(x, \psi) = 2^{-\frac{E(h(x))}{c(\psi)}} \quad (6)$$

$c(n)$ is the expected average path length for a randomly sub-sampled dataset of size n, computed using the same harmonic formula. This term serves as a normalization baseline, which is the average isolation difficulty one would expect if all points were equally easy to separate, with no anomalies present. Dividing by $c(n)$ scales the score consistently between 0 and 1.

The score is interpreted as follows. A score approaching 1 indicates that the data point was isolated very quickly across most trees, meaning it is a strong anomaly candidate. A score near 0.5 suggests an average path length close to the expected baseline, indicating the point is a normal instance. A score approaching 0 indicates the point was deeply embedded in the data and required an unusually large number of splits to isolate, marking it as a very typical, well-clustered observation. Since the score's primary purpose is to impose a ranking on the dataset, the authors suggest sorting the data and taking the first m instances as anomalies, corresponding to a percentage threshold (e.g. the top 1% of anomaly scores can be taken as anomalies).

In practice, a threshold is applied to the anomaly score to produce a binary classification decision. Data points whose score exceeds the threshold are labeled as anomalies; those below are considered normal. The appropriate threshold depends on the application domain and the acceptable trade-off between false positives and false negatives and is typically tuned based on the expected contamination rate in the dataset.

III. Implementation

This section explains how the algorithm was implemented on a data set consisting of rare bird sightings.

A. Dataset

The eBird database [3], which contains more than 2 billion observations of birds, was selected to implement the iForest algorithm. This dataset differs from those used by [1] in that it involves some categorical and non-continuous attributes. These non-continuous attributes cannot be used without some transformation, since the algorithm depends on making random splits across a continuous range of integers or real numbers. However, it is suitable in that it contains ground truth anomaly labels. These labels are supplied via eBird's rigorous expert review system, where any observation that is flagged as unlikely is reviewed by an expert, who then decides whether the observation is legitimate or not. eBird's database contains an attribute *Reviewed*, which has the value "1" in cases an observation was flagged for review. Observations that were rejected are marked as "0" in another attribute, *Approved*. We thus filter out all observations where *Approved=0*, and set observations with *Reviewed=1* as the ground-truth for anomalies, and those with *Reviewed=0* as normal observations.

A given observation is flagged as requiring review based on particular combinations along four dimensions:

1) Location: One observation may be normal in one location but not in another;

2) Species: What is a normal observation for one species, may be abnormal for another;

3) Time of year: Since birds migrate seasonally, a winter observation may be normal, but a corresponding observation in summer may be abnormal.

4) Count: A count of 500 birds in a single observation may be normal in some cases, but highly irregular in many others.

This database offers a very clear criteria then of what constitutes an anomaly: Combinations of these above four dimensions map to normal or abnormal bird observations. This directly informs us of the four features that are needed in

training the algorithm. iForest's job is then to learn which points in this four-dimensional space are anomalies.

The actual dataset used here was filtered to make it practical to run on a personal computer, while still being "massive". As a result, when downloading from eBird.org using the standard UI filter tool, we filtered observations only within British Columbia, taken from Jan. 2016, to Jan. 2026. We included sampling event data, but did not include unvetted data. All species were included.

In addition, a number of decisions to filter the dataset further were made to simplify the learning process. These are summarized below using the original attribute names in the eBird database:

- An observation was retained only if CATEGORY == "species". Observations are normally classified by species; however in some cases, it is possible to classify by subspecies or even genus if a birder is unsure. We only retained species-level observations.
- An observation was retained only if EXOTIC CODE == "" or "N". Species that are escaped and not native to British Columbia were excluded. "N" refers to species that were introduced but are now well established in the wild. Our initial decision to keep these resulted in the model learning that all observations of species with EXOTIC CODE == "N" were anomalies, a kind of success, since these species *are* anomalous in that they tend to live in local ranges clustered around human development, and in rather large numbers. However, we decided to re-do the experiment with EXOTIC CODE == "N" observations all removed, which is what we report here. This provided a test of finding true anomalies among native species.
- An observation was retained only if OBSERVATION TYPE == "Traveling" or "Stationary". This filter is recommended by eBird since other values involve unusual birding observations.
- An observation was retained only if ALL SPECIES REPORTED == 1. When it has a value of 0, this indicates unreliable observations.
- Checklists that involve large distances traveled may involve errors where a birder may have neglected to turn off their app before driving away in their vehicle. This would render the location data meaningless. As a result, we followed eBird's recommendation to filter out any EFFORT DISTANCE KM $\leq$ 10. Likewise, DURATION MINUTES was recommended to be filtered to $\leq$ 300 for the same reasons.
- We did not use or filter on TIME OBSERVATIONS STARTED. This was a candidate for feature inclusion in fact, since some species are nocturnal (owls for example). However, eBird's reviewing isn't flagged by time of day, and so we ignored this attribute since it didn't affect our ground truth values.

Following these curation steps, the dataset contained 1,093,203 observations with 8,161 (0.747% of the total data) labeled as anomalies.

This dataset provides an interesting test case since the model necessarily must learn to distinguish between species, a categorical attribute. The original authors omitted any attributes that were not continuous; however, we attempted to transform such attributes to continuous ones.

B. Feature Transformations

Because isolation forest splits along a range of integers with extremes landing on a scale, categorical features like *Species* cannot be used, as mentioned above. While one-hot encoding is a possible way forward, it creates excessively high-dimensional sparse feature spaces which degrade the performance of algorithms like Isolation Forests [4]. Instead, we would like to convert our species labels to some kind of meaningful numerical feature. We did this by assigning each species a relative frequency, using Target-Agnostic encoding [5]. We simply count the number of rows for that species and divide by the number of total rows in the dataset. When Isolation Forest tries to split by species, it will split the species into more common and less common ones, which suits our goal of finding anomalies. However, the continuum of species by frequency will obviously not mirror the way in which species tend to cluster by location, day and count. Another possibility would have been to limit the dataset to a small set of species or even a single very frequent species; the model would undoubtedly perform better in this case, but it also provides a less interesting test.

Whereas location and count features are beautifully handled directly as numerical values via *Latitude*, *Longitude* and *Observation Count*, *Time of Year* requires transformation. As we've seen, isolation trees work by randomly selecting a feature and drawing a split between its minimum and maximum values. Simple dates will not capture the seasonal nature of bird migration. Likewise, if we represent the day of the year as a linear integer (1 to 365), the algorithm will treat day 1 and day 365 as maximally distant, failing to recognize their temporal adjacency. Dates are cyclical and so we used *sine* and *cosine* transformations [5, 6], yielding two features, day_{sin}, day_{cos}. Together, these values range from –1 to 1:

$$day_{sin} = \sin\left(\frac{2\pi \cdot day_{of_{year}}}{365}\right) \tag{7}$$

$$day_{cos} = \cos\left(\frac{2\pi \cdot day_of_year}{365}\right) \tag{8}$$

Using the *sine* and *cosine* transformations as features instead, days that are maximally distant in the calendar year will have maximally distant feature values, and January 1st will be adjacent to December 31st, as required.

C. Configuration and Data Loading

To replicate the experiments presented in the original research paper, we have set the following parameters in a centralized configuration class for the entire pipeline.

```
@dataclass
class IForestConfig:
    input_csv: str =
        "data/ebird_data_processed_sampled.csv"
    n_trees: int = 100
    sample_size: int = 256
    random_state: int = 42
    anomaly_threshold: float = 0.60
    tree_grid: tuple = (25, 50, 100, 200)
    sample_grid: tuple = (2, 4, 8, 16, 32, 64,
        128, 256, 512,
        1024, 2048, 4096)
```

Before training the model, we filtered the orginal data down to inculde the following six features: species_frequency, OBSERVATION COUNT, LATITUDE, LONGITUDE, day_sin, day_cos, and the ground truth label of REVIEWED.

```
FEATURE_COLUMNS = [
    "species_frequency",
    "OBSERVATION COUNT",
    "LATITUDE",
    "LONGITUDE",
    "day_sin",
    "day_cos"
]
LABEL_COLUMN = "REVIEWED"

df = load_processed_data(config.input_csv)
X = get_feature_matrix(df)
y = get_labels(df)
```

D. Data Processing in PySpark

We applied data cleaning techniques such as trimming column names and handling missing values (Observation count == “X”) to convert the eBird raw data text file into a numerical format using PySpark.

```
df = (
   df.withColumn("OBSERVATION COUNT",
      when(col("OBSERVATION COUNT").isNull() |
           (col("OBSERVATION COUNT") == "") |
           (col("OBSERVATION COUNT") == "X"),
           None)
   .otherwise(col("OBSERVATION COUNT"))
   .cast("double"))
   .withColumn("LATITUDE",
      when(col("LATITUDE").isNull() |
           (col("LATITUDE") == ""), None)
   .otherwise(col("LATITUDE")).cast("double"))
   .withColumn("LONGITUDE",
      when(col("LONGITUDE").isNull() |
           (col("LONGITUDE") == ""), None)
   .otherwise(col("LONGITUDE")).cast("double"))
)
```

To reduce the size of our original dataset, we only consider relevant records where CATEGORY == "species", OBSERVATION TYPE is either "Traveling" or "Stationary", ALL SPECIES REPORTED == 1, DURATION MINUTES <= 300, and EFFORT DISTANCE KM <= 10 or null.

```
filtered = (
   df.filter(col("CATEGORY") == "species")
      .filter(col("EXOTIC CODE").isNull() |
 (col("EXOTIC CODE") == ""))
      .filter(col("OBSERVATION
         TYPE").isin("Traveling", "Stationary"))
      .filter(col("ALL SPECIES REPORTED") == 1)
      .filter(col("DURATION MINUTES") <= 300)
      .filter(col("EFFORT DISTANCE KM").isNull()
         | (col("EFFORT DISTANCE KM") <= 10))
      .filter(col("TAXON CONCEPT
         ID").isNotNull())
)
```

The observation date is converted into a seasonal cyclic format using the sine and cosine functions for the day of the year to preserve the annual cycle of bird migration.

```
filtered = (
filtered.withColumn("OBSERVATION DATE",
   to_date(col("OBSERVATION  DATE"),
         "yyyy-MM-dd"))
      .filter(col("OBSERVATION
         DATE").isNotNull())
      .withColumn("day_of_year",
         dayofyear(col("OBSERVATION DATE")))
      .withColumn("day_sin",
         sin(col("day_of_year") * lit(2 *
         math.pi / 365.0)))
      .withColumn("day_cos",
         cos(col("day_of_year") * lit(2 *
         math.pi / 365.0)))
)
```

For each TAXON CONCEPT ID, we count the number of corresponding observations and divide them by the total number of filtered rows, producing the species_frequency feature.

```
species_counts = filtered.groupBy("TAXON
         CONCEPT ID")
      .agg(spark_count("*")
      .alias("species_count")
)

total_count = species_counts.agg(
   spark_sum("species_count").alias("total)
).collect()[0]["total"]

species_freq =
   species_counts.withColumn("species_frequency"
         ,
   col("species_count") /lit(float(total_count))
).select("TAXON CONCEPT ID",
         "species_frequency")
```

E. *Stratified Sampling*

To avoid any potential performance issues for our experiments, we reduced the original dataset by around 95% using quantiles on the species_frequency column, while maintaining the original distribution of the species. This was required because of the large number of species, and the wide range of frequencies they occupied. We did not want to risk throwing away observations of less frequent species, which are more likely to be anomalies.

```
num_quantile_bins = 8
quantile_probs = [i / num_quantile_bins for i
        in range(num_quantile_bins + 1)]
boundaries =
        df.approxQuantile("species_frequency",
           quantile_probs,
           0.001)
sample_fraction = 0.05
sampled = (
   grouped.withColumn("u", rand(random_seed))
      .filter(col("u") < sample_fraction)
      .drop("u")
      .cache()
)

freq_bin = None
    for i in range(len(boundaries) - 1):
        lower = boundaries[i]
        upper = boundaries[i + 1]
        label = f"Q{i + 1}"
        if i == 0:
            condition =
         (col("species_frequency") >= lower) &
         (col("species_frequency") <= upper)
        else:
            condition =
         (col("species_frequency") > lower) &
         (col("species_frequency") <= upper)

        if freq_bin is None:
            freq_bin = when(condition, label)
        else:
            freq_bin = freq_bin.when(condition,
         label)
```

Figure 1 displays the output summary for each of the quantile bins after sampling our processed data.

```
Original subgroup distribution:
+--------+-------+
|freq_bin|count  |
+--------+-------+
|Q1      |2746040|
|Q2      |2822288|
|Q3      |2718881|
|Q4      |2735962|
|Q5      |2953302|
|Q6      |2543182|
|Q7      |3113192|
|Q8      |2230449|
+--------+-------+

Sampled subgroup distribution:
+--------+------+
|freq_bin|count |
+--------+------+
|Q1      |137130|
|Q2      |140865|
|Q3      |135736|
|Q4      |136889|
|Q5      |148214|
|Q6      |127249|
|Q7      |155494|
|Q8      |111626|
+--------+------+
```

Figure 1:Sampling Distribution

F. *Isolation Tree Construction*

We define the model of the Isolation Tree to consist of Internal and External nodes. An internal node contains the selected split attribute, split value, and its left and right children, and an external node would have the number of instances that would end there.

```
@dataclass
class ExternalNode:
   size: int

@dataclass
class InternalNode:
   split_att: int
   split_value: float
   left: object
   right: object
```

Each recursive process involves choosing, uniformly at random, one such feature out of all those that have non-constant values in the current node. Splitting involves drawing a splitting point uniformly at random within the range of values taken by this particular feature. If the value of the feature in an example is less than this split value, the example goes left; otherwise, it goes right. Recursive processes stop when the depth of the tree reaches a certain limit or if there are no more examples in the node.

```
split_att =
        int(self.rng.choice(valid_features))
min_val = float(col_mins[split_att])
max_val = float(col_maxs[split_att])
split_value = float(self.rng.uniform(min_val,
        max_val))

left_mask = X[:, split_att] < split_value
right_mask = ~left_mask
```

The calculation of path length starts from the root node to the leaf node of the tree. If there are multiple occurrences in the leaf node, the path length is adjusted by the *c(n)* value from the Isolation Forest research paper, which uses the c_factor(n) function.

```
def _path_length(self, x, node,
        current_height):
   if isinstance(node, ExternalNode):
      if node.size <= 1:
         return float(current_height)
      return float(current_height) +
         c_factor(node.size)
```

G. *Isolation Forest Ensemble*

IsolationForestCustom constructs the forest using Isolation Trees. In building each tree, a sub-sampling step is carried out where ψ elements are selected randomly without replacement from the complete feature matrix. The height of the tree is also limited as described in the paper.

```
actual_sample_size = min(self.sample_size,
        n_samples)
self.height_limit=math.ceil(
   math.log2(max(actual_sample_size, 2))
)

indices = self.rng.choice(n_samples,
        size=actual_sample_size, replace=False)
sample = X[indices]
tree = IsolationTree(self.height_limit,
        tree_rng).fit(sample)
self.trees.append(tree)
```

Once all the decision trees have been built, the average distance between all observations is calculated within the forest. Next, the anomaly score is obtained by applying the scoring function to the calculated value.

```
def score_samples(self, X):
   mean_path_lengths = self._mean_path_length(X)
   return np.array(
      [anomaly_score(path_length,
         self.effective_sample_size) for
         path_length in mean_path_lengths],
         dtype=float
   )
```

Afterwards, we calculate the anomaly prediction scores and classify each sample based on the threshold value of 0.6.

```
def predict(self, X, threshold=0.60):
   scores = self.score_samples(X)
   return (scores >= threshold).astype(int)
```

IV. Results

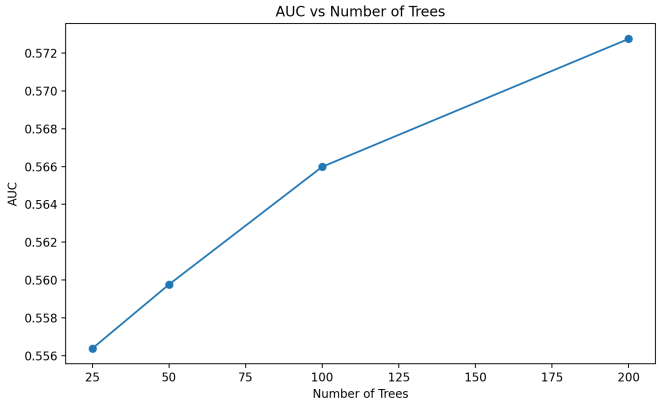


Figure 2:AUC vs. number of trees, t

In Figure 2, we can see that the accuracy rose from approximately 0.556 to 0.573 as the number of trees rose from 25 to 200. Nonetheless, there is not much progress, showing that adding more trees will make only minimal improvements.

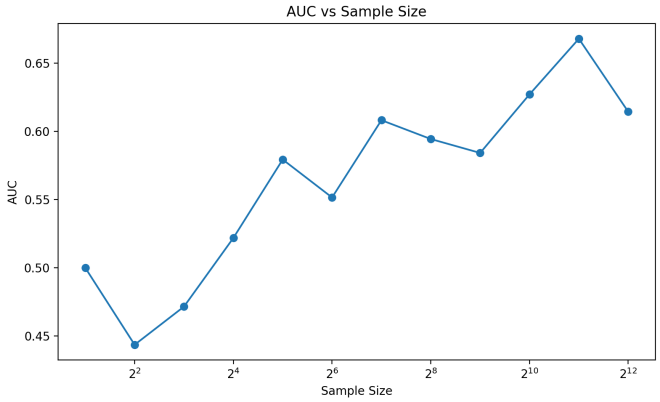


Figure 3:AUC vs. subsample size, ψ

Figure 3 shows that the AUC improves as the sample size increases, peaking at around 0.67 with a sample size of 2^{11}.

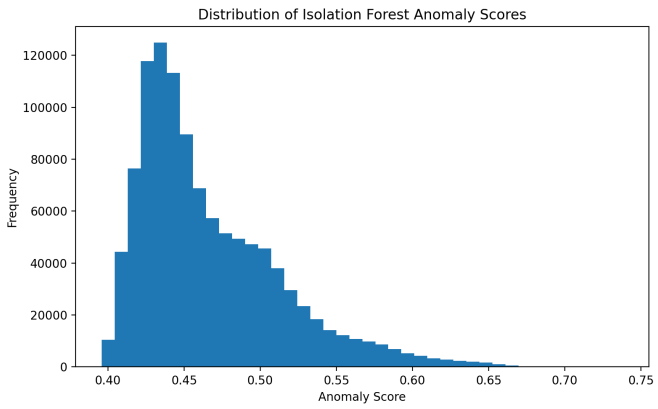


Figure 4:distribution of Isolation Forest anomaly scores

Figure 4 suggests that the distribution of anomaly scores is positively skewed where most of the scores fall within the range of 0.40 to 0.50 with an extended tail on the higher side corresponding to anomalies.

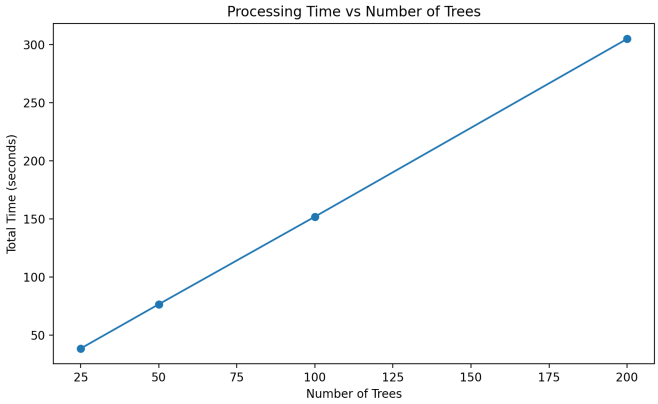


Figure 5:Processing time vs. number of trees, t

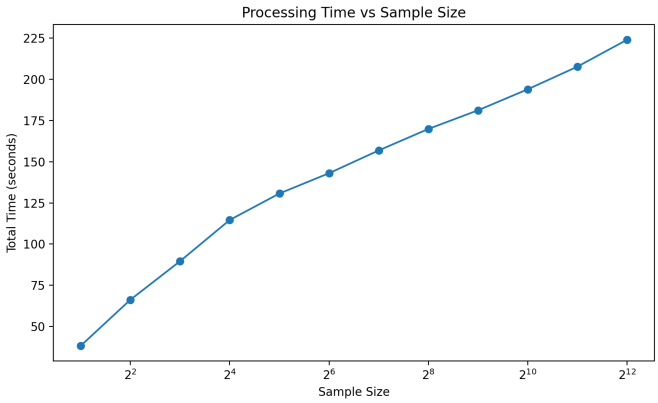


Figure 6:Processing time vs. subsample size, ψ

For the runtime graphs shown in Figures 5 and 6, we conclude that the amount of time required is linear with respect to the number of trees and the number of samples used. Though using more trees and more samples helps in improving the model's performance, the runtime requirement for training the model is increased by a significant margin when compared to the small gain in the AUC value.

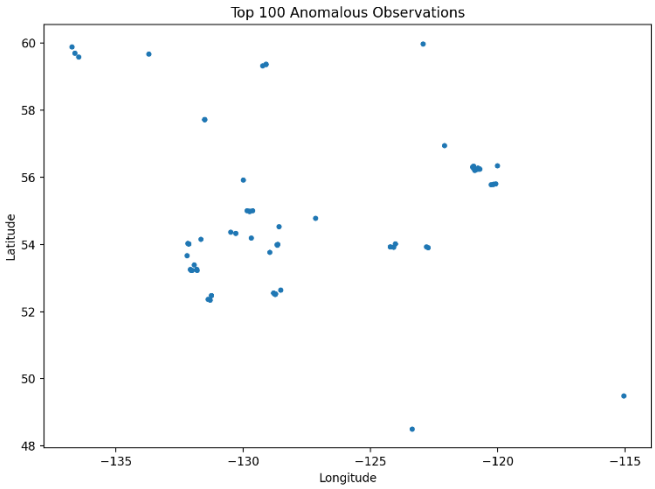


Figure 7:Top one-hundred anomaly scores by location

Based on Figure 7, which shows the top 100 anomalous observations, we conclude that there were more anomalies in northern British Columbia, indicating the need to further investigate the bird species found in that area.

V. Discussion

The rather low AUC score of 0.67 indicated that anomaly detection isn't accurately matching the ground truth labels consistently. Figure 7 suggested that the splits on latitude and longitude may not be working well. This is especially true since most of the population of British Columbia lives in the lower half of the province, while most anomalies detected by the model are in the northern half. Since Isolation Forest randomly chooses values to split on, it is more likely to isolate observations based on location in less densely populated regions.

To explore this possibility, Figure 8 plots our top one-hundred anomaly scores overlaid on a map of British Columbia. To visualize density by region, a heatmap that visually shows observation density is overlaid for comparison. Hexagonal bins were computed with 75 hexagons spanning the full range of latitudes. This meant that each hexagon covers an area approximately 38-40 km wide (east-west) and about 44 km tall (north-south). Total number of observations is coded by colour with dark blue for dense regions and light blue for sparse regions, on a log-base-10 scale. Yellow stars display correctly classified anomalies, while red circles display incorrect predictions. The size of the circle scales with the number of anomalies detected in a single hexagon.

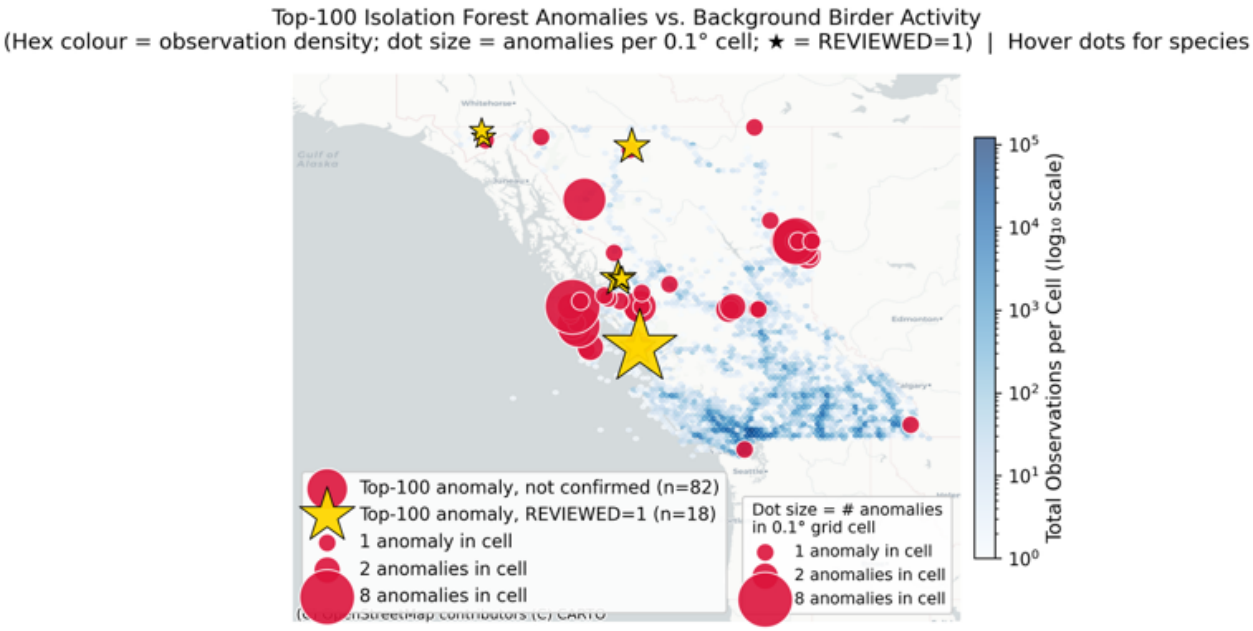


Figure 8: Top 100 anomalies plotted by location with observation density displayed

Figure 8 shows that only a few anomalies were detected from the densely populated southern regions, with most occurring in the sparsely populated northern half of the province.

As a follow-up, we explored which regions had higher than average anomaly rates. Figure 9 visualizes the spatial distribution of ground-truth anomaly rate relative to the global baseline of 0.75% across British Columbia. For each hexagonal cell, the mean per-observation deviation from the expected rate is computed as:

$$deviation = (REVIEWED = 1) - 0.747\% \quad (6)$$

Cells are coloured using a diverging red-blue scheme: red indicates an above-average true anomaly rate, while blue indicates below-average rates. Cells containing fewer than 20 observations were omitted. Overlaid are the top 100 model-flagged anomalies with green dots indicating model predictions that were not ground truth anomalies and yellow dots for observations that were correctly labeled as anomalies

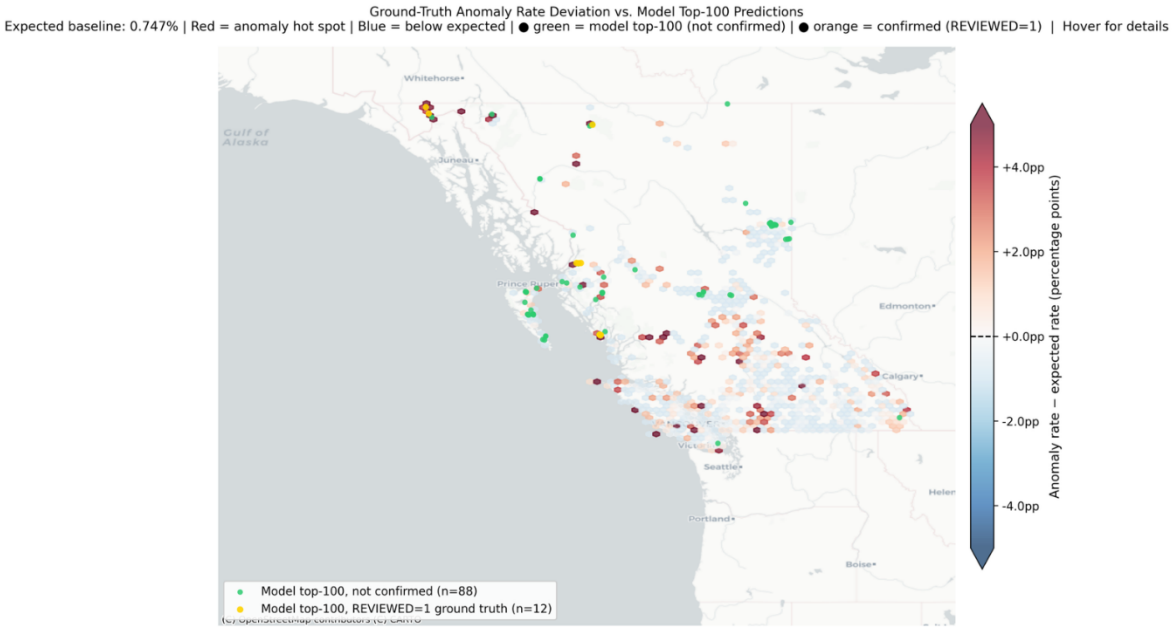


Figure 9: Spatial deviation of ground-truth anomaly rate from the global baseline (0.747%) across British Columbia

Figure 9 shows that regions with high anomaly rates are only correctly discovered in a few regions in the north-west, where the observation density is very low. Anomalies in the south were invariably not discovered.

This observation can be explained by considering the way the Isolation Forest algorithm works. The algorithm is blind to the fact that the data is not equally distributed among the latitude and longitude features. It is equally likely to choose any latitude or longitude between the minimum and maximum values. As a result, the observations taken from northern British Columbia are more likely to be isolated early. On the other hand, observations from southern British Columbia, and from regions with high density of observations. This explains why almost no anomalies were discovered in those regions.

To address this situation, the latitude and longitude data could be transformed into ranks. Splitting on a ranked value would ensure that splits would happen more often in densely populated areas, thus allowing the algorithm to find patterns in those regions more easily. A second solution would be to sample the dataset such that local bins are created (like the ones plotted in figures 8 and 9), with an equal number of data points drawn from each bin. This could have been done alongside the species-specific stratified sampling, for example.

The Isolation Forest algorithm is thus sensitive to features with skewed distributions, where it is biased away from finding anomalies in dense regions and towards finding them in sparse regions. This is a kind of feature-specific swamping effect that the authors described, where one feature (location in this case) is swamped, preventing successful learning from other features in those regions.

In addition, while we mostly discussed the location features here, a full review of the algorithm's performance on this dataset would similarly explore the other features of time, count, and species. We transformed the categorical feature Species into a numerical one by listing each species by its frequency in the dataset. This allowed more frequently seen species (less likely to be an anomaly) to cluster together and more rare species (more likely to be an anomaly) to do the same. However, this is an admittedly crude treatment. It ignores the complex way in which an anomaly can be defined. Frequency would ignore factors like seasonal rarity, where some species are anomalous only in winter or summer. It also ignores the fact that many species are more easily observed due to their habits or preferred habitat. A model that focuses on a carefully selected group of species that takes the above into account, or even just a single species would likely be more successful.

VI. Conclusion

The Isolation Forest algorithm was successfully implemented via PySpark to learn anomalies among the eBird database, which lists bird observations. An AUC of 0.67 was achieved, and processing speed was confirmed to increase linearly as the number of trees and subsamples increased.

Closer inspection of the results showed that most of the anomalies were detected at latitudes and longitudes with very few observations; the model did not predict anomalies in regions with many observations – it focused primarily on sparse regions. This illustrated a shortcoming of the Isolation Forest algorithm – a tendency for feature-specific swamping: If a single feature has a skewed distribution across its range, the algorithm will tend to find anomalies only from the sparse part of that feature's range, ignoring other features' effects in the dense part of the range. As a result, anomalies were neither predicted nor detected among the anomalous observations from densely populated regions. Future work to address this issue is needed, with a few promising ideas being the use of a rank transformation of the feature or making subsampling sensitive to feature density.

Acknowledgment

Thank you to Professor Alex Thomo for teaching us how to work with Systems for Massive Datasets.

References

[1] F. T. Liu, K. M. Ting and Z. -H. Zhou, “Isolation Forest,” Eigth IEEE International Conference on Data Mining, Pisa, Italy, 2008, pp. 413–422, doi: 10.1109/ICDM.2008.17.

[2] M. Zaharia, M. Chowdhury, T. Das, A. Dave, J. Ma, M. McCauley, M.J. Franklin, S. Shenker, and I. Stoica, "Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing," in *Proceedings of the 9th USENIX Conference on Networked Systems Design and Implementation (NSDI)*, 2012, pp. 15–28.

[3] eBird, "eBird: An online database of bird distribution and abundance [web application]," Cornell Lab of Ornithology, Ithaca, NY, 2026. [Online]. Available: https://ebird.org. [Accessed: March 20, 2026].

[4] K. Potdar, T. S. Pardawala, and C. D. Pai, "A comparative study of categorical variable encoding techniques for neural network classifiers," *Int. J. Comput. Appl.*, vol. 175, no. 4, pp. 7–9, Oct. 2017.

[5] F. Pargent, F. Pfisterer, J. Thomas, and B. Bischl, "Regularized target encoding outperforms traditional methods in supervised machine learning with high cardinality features," *Comput. Stat.*, vol. 37, no. 5, pp. 2671–2692, Nov. 2022.

[6] T. Mahajan, G. Singh, and G. Bruns, "An experimental assessment of treatments for cyclical data," in *Proc. Comput. Sci. Conf. CSU Undergraduates*, Virtual, 2021.

[7] A. Ramachandran, S. Chatterjee, T. F. B. Neergaard, M. Oberndoerfer, A. Maier, and S. Bayer, "A deep learning framework for heat demand forecasting using time-frequency representations of decomposed features," *Energy and AI*, vol. 24, p. 100704, May 2026.